

ROSTR: A Unified Architecture for Production-Grade Multi-Agent Systems with Phase-Aware Orchestration and Persistent Knowledge Compounding

****Patrick Diamitani****

Independent Researcher

patrick@diamitani.com

****April 2026****

Abstract

Modern multi-agent AI systems face four fundamental challenges: (1) the prompting bottleneck—translating user intent into precise agent instructions requires expertise most users lack; (2) retrieval brittleness—agents perform shallow, single-pass information gathering without source validation or gap detection; (3) context loss—agents operate statelessly, losing knowledge across sessions and failing to compound organizational learning; and (4) naive task routing—orchestration systems route work by keyword matching rather than workflow phase or priority scoring.

We present ROSTR (Runtime, Orchestration, State, Tools, Reference), a modular agent operating system that addresses these challenges through four integrated components: PAL (Prompt Abstraction Layer) compiles natural language intent into structured agent manifests using a five-stage pipeline; RAG DAL (Retrieval-Augmented Generation Dynamic Acquisition Layer) performs autonomous multi-pass web retrieval with hierarchical source credibility scoring; NPAO (Navigate, Prioritize, Allocate, Orchestrate) introduces a 5D phase taxonomy and multi-dimensional priority scoring for context-aware task routing; and the Rostr Hub provides a persistent, multi-namespace knowledge architecture enabling cross-session and cross-agent context preservation.

We detail the technical mechanisms of each component, provide measurable specifications, position the work against existing agent frameworks (LangChain, CrewAI, AutoGPT, Bee Framework), and outline an empirical validation framework. Early architectural implementations demonstrate the system's viability for GTM operations, though comprehensive empirical validation remains future work. The framework is designed as open-source infrastructure to enable production-grade multi-agent systems with reduced brittleness, improved knowledge persistence, and phase-aware workflow management.

****Keywords:**** multi-agent systems, agent orchestration, retrieval-augmented generation, prompt engineering, knowledge management, workflow automation

1. Introduction

1.1 Motivation

The rapid advancement of large language models (LLMs) has enabled sophisticated single-agent systems, but deploying **multi-agent teams** for complex, multi-phase work remains fragile. Four systemic problems persist:

The Prompting Bottleneck. Agent performance is gated by prompt quality. Users must understand model capabilities, craft precise instructions, and translate domain intent into LLM-compatible formats. This expertise gap limits who can effectively deploy agents [1, 2].

Retrieval Brittleness. Standard RAG implementations perform single-pass retrieval without source quality assessment or coverage validation. Agents cannot distinguish authoritative sources from unreliable ones, nor detect when retrieved information is insufficient [3, 4].

Context Loss Across Sessions. Agents operate statelessly—each session starts from scratch. Decisions made, knowledge gathered, and workflows established in prior sessions are lost. There is no organizational memory, no knowledge compounding [5].

Naive Task Routing. Orchestrators route tasks by keyword matching ("code" → builder agent) without considering workflow phase (research vs. production debugging), priority (revenue-impacting vs. internal tooling), or dependency chains (what must finish before this starts) [6, 7].

These problems compound: poor prompts yield poor agent instructions; brittle retrieval yields unreliable outputs; context loss forces repeated work; naive routing executes the right tasks in the wrong order.

1.2 Contribution

We present ROSTR, a unified architecture for multi-agent systems that addresses each challenge through modular, composable components:

1. **PAL (Prompt Abstraction Layer)** — A compiler-inspired pipeline that transforms natural language intent into structured agent runtime manifests through intent extraction, context injection, semantic enhancement, runtime compilation, and deterministic routing.
2. **RAG DAL (Dynamic Acquisition Layer)** — An autonomous multi-pass retrieval system with three-tier source credibility scoring (academic/authoritative, editorial/verified, community/UGC), self-assessed coverage validation, and persistent knowledge base ingestion with provenance tracking.

3. ****NPAO (Navigate, Prioritize, Allocate, Orchestrate)**** — A decision engine introducing the ****5D phase taxonomy**** (PreD, Design, Development, Deployment, Debugging), multi-dimensional priority scoring (phase urgency, dependency impact, business impact, resource efficiency), and phase-aware agent allocation.

4. ****Rostr Hub**** — A persistent, multi-namespace knowledge platform providing agent registration, state management (session/project/organization/agent levels), cross-agent communication protocols, and shared reference architecture.

5. ****System Integration**** — Architectural patterns for composing these components into production systems, with reference implementation for GTM operations demonstrating real-world viability.

1.3 Key Innovations

Our primary contributions are:

****Architectural:**** First framework unifying intent compilation, hierarchical credibility-weighted RAG, phase-aware orchestration, and persistent multi-namespace knowledge architecture in a single system.

****Algorithmic:****

- Multi-dimensional priority scoring with configurable phase/dependency/business/resource weights
- Autonomous multi-pass retrieval with convergence criteria based on confidence thresholds and source cross-validation
- Intent compilation pipeline mapping natural language → typed agent manifests
- Phase-aware agent allocation considering capability, context, and load

****Conceptual:****

- ****5D Phase Taxonomy**** with formalized PreD (Pre-Development) phase—structuring the research and feasibility work that precedes design
- ****Three-tier source credibility hierarchy**** for retrieval quality control
- ****Multi-namespace knowledge persistence**** enabling organizational context compounding
- ****Agent manifest as infrastructure-as-code**** for reproducible, versionable assistant definitions

****Empirical Validation Framework:**** We outline measurable hypotheses and experimental designs for validating each component's efficacy, though full empirical validation remains future work.

1.4 Paper Organization

Section 2 reviews related work in agent orchestration, RAG systems, and workflow management. Section 3 presents the unified architecture. Sections 4-7 detail PAL, RAG DAL,

NPAO, and Rostr Hub technical specifications. Section 8 discusses integration patterns and reference implementations. Section 9 outlines the empirical validation framework. Section 10 discusses limitations, future work, and broader implications. Section 11 concludes.

2. Related Work

2.1 Multi-Agent Orchestration Frameworks

LangChain [8] pioneered modular agent construction through tool chains and memory abstractions. However, it lacks: (1) declarative agent specifications—developers write imperative Python code rather than compiling from high-level intent; (2) persistent state—memory modules exist but aren't architected for cross-session or cross-agent knowledge sharing; (3) phase-aware routing—task delegation uses keyword matching or manual orchestration logic.

CrewAI [9] introduced role-based agent teams with delegation patterns. Agents have defined roles (researcher, writer, analyst) and can delegate subtasks. Limitations: (1) no formalized workflow phase taxonomy—all work treated uniformly regardless of whether it's exploratory research vs. production deployment; (2) no built-in retrieval quality control—RAG implementations are external; (3) limited state persistence—primarily session-based.

AutoGPT [10] demonstrated autonomous agent execution with self-directed task decomposition. However: (1) unstructured exploration—no phase gates or completion criteria; (2) no priority model—tasks executed in generation order rather than business priority; (3) minimal human oversight—hard to enforce approval gates for high-stakes actions.

MetaGPT [11] applied software engineering roles (product manager, architect, engineer) to agent teams. Introduces standardized output artifacts (PRDs, design docs, code) but lacks: (1) persistent knowledge architecture across projects; (2) retrieval quality control; (3) dynamic priority-based routing.

Bee Framework (IBM) [12] provides enterprise agent infrastructure with strong tool integration. However: (1) closed-source limits inspection and modification; (2) heavier deployment footprint; (3) less emphasis on phase-aware workflow management.

DOE Pattern [13] (Directives → Orchestration → Execution) provides a mental model for agent architecture. ROSTR extends DOE by: (1) adding PreD phase before directives; (2) persistent state across D→O→E cycles; (3) modular open-source implementation.

ROSTR's contribution vs. these systems: **unified architecture** combining compilation, credibility-weighted retrieval, phase taxonomy, and persistent hub in a single framework with measurable specifications.

2.2 Retrieval-Augmented Generation

Standard RAG [14, 15] performs: query → retrieve documents → rank by embedding similarity → augment LLM context → generate. Limitations:

1. **Single-pass retrieval** — No iterative refinement based on coverage gaps
2. **No source quality control** — Reddit comments ranked equally with peer-reviewed papers
3. **No self-assessment** — System doesn't detect when retrieved information is insufficient or contradictory

Advanced RAG systems [16, 17] introduce query rewriting, multi-step retrieval, and re-ranking. RAG DAL extends this with:

1. **Three-tier credibility hierarchy** — Explicit source stratification (academic/editorial/community) with credibility weights (1.0/0.75/0.40)
2. **Autonomous multi-pass loop** — Iterative retrieval with convergence criteria (confidence $\geq 8/10$ per sub-topic, minimum 2 Tier 1/2 source confirmation, no contradictions)
3. **Gap detection** — Explicit identification of uncovered sub-topics triggering targeted re-search
4. **Persistent knowledge base** — Ingestion of validated findings into shared repository with provenance tracking

Closest work: **STORM** [18] performs multi-perspective question generation for Wikipedia-style articles. RAG DAL differs: (1) hierarchical credibility vs. uniform sourcing; (2) confidence thresholds for convergence vs. fixed retrieval count; (3) cross-agent persistence vs. single-article scope.

2.3 Workflow and Task Management

Workflow engines (Airflow [19], Prefect [20], Temporal [21]) excel at orchestrating deterministic DAGs but assume: (1) predefined task structure—human defines all steps upfront; (2) no dynamic priority adjustment—execution order fixed; (3) no agent-specific context (tools, capabilities, phase specialization).

Issue trackers (Jira, Linear) provide priority scoring (P0-P4) but lack: (1) multi-dimensional weighting (phase + dependency + business + resource); (2) agent capability matching; (3) automated phase classification.

NPAO contributes: **5D phase taxonomy** formalizing the workflow lifecycle from research (PreD) through debugging, plus **multi-dimensional scoring** for context-aware priority calculation.

2.4 Prompt Engineering and LLM Compilers

DSPy [22] optimizes prompts through automated refinement but focuses on **response quality optimization** rather than **agent manifest generation**. PAL's compilation target is a structured runtime configuration (tools, permissions, memory policy), not an optimized prompt string.

Guidance [23] and **LMQL** [24] provide structured generation languages for LLM output formatting. PAL operates at a higher abstraction layer: natural language intent → complete agent specification, encompassing system instructions, tool schemas, memory policies, and deployment configs.

LangSmith prompt hub [25] provides versioned prompt templates. PAL's enhancement pipeline transforms user intent (potentially vague or incomplete) into precise, context-injected, domain-enhanced instructions—a compilation process rather than template selection.

2.5 Knowledge Management and Memory

Vector databases (Pinecone, Weaviate, Chroma) provide embedding storage and similarity search. ROSTR's Reference Hub extends this with: (1) **multi-namespace hierarchy** (project/org/team/global); (2) **structured metadata** (decisions, learnings, artifacts, not just embeddings); (3) **cross-agent access patterns**—any agent in a namespace can query the shared knowledge base.

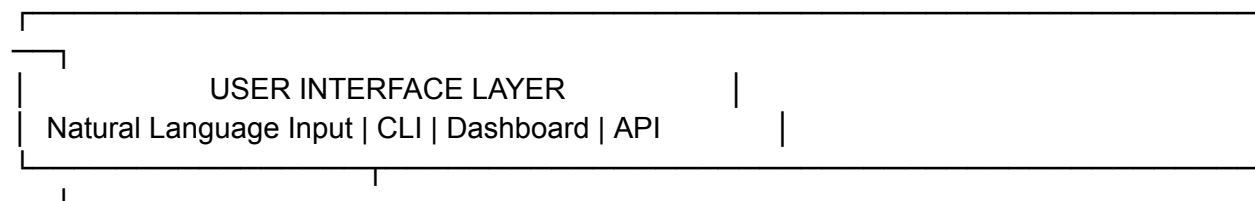
Agent memory systems [26, 27] typically provide: short-term (conversation buffer), long-term (vector store of past interactions). ROSTR's state architecture adds: (1) **four-level hierarchy** (session/project/org/agent); (2) **knowledge compounding**—research findings from one agent/session available to others; (3) **provenance tracking**—what was learned, when, from what source, with what confidence.

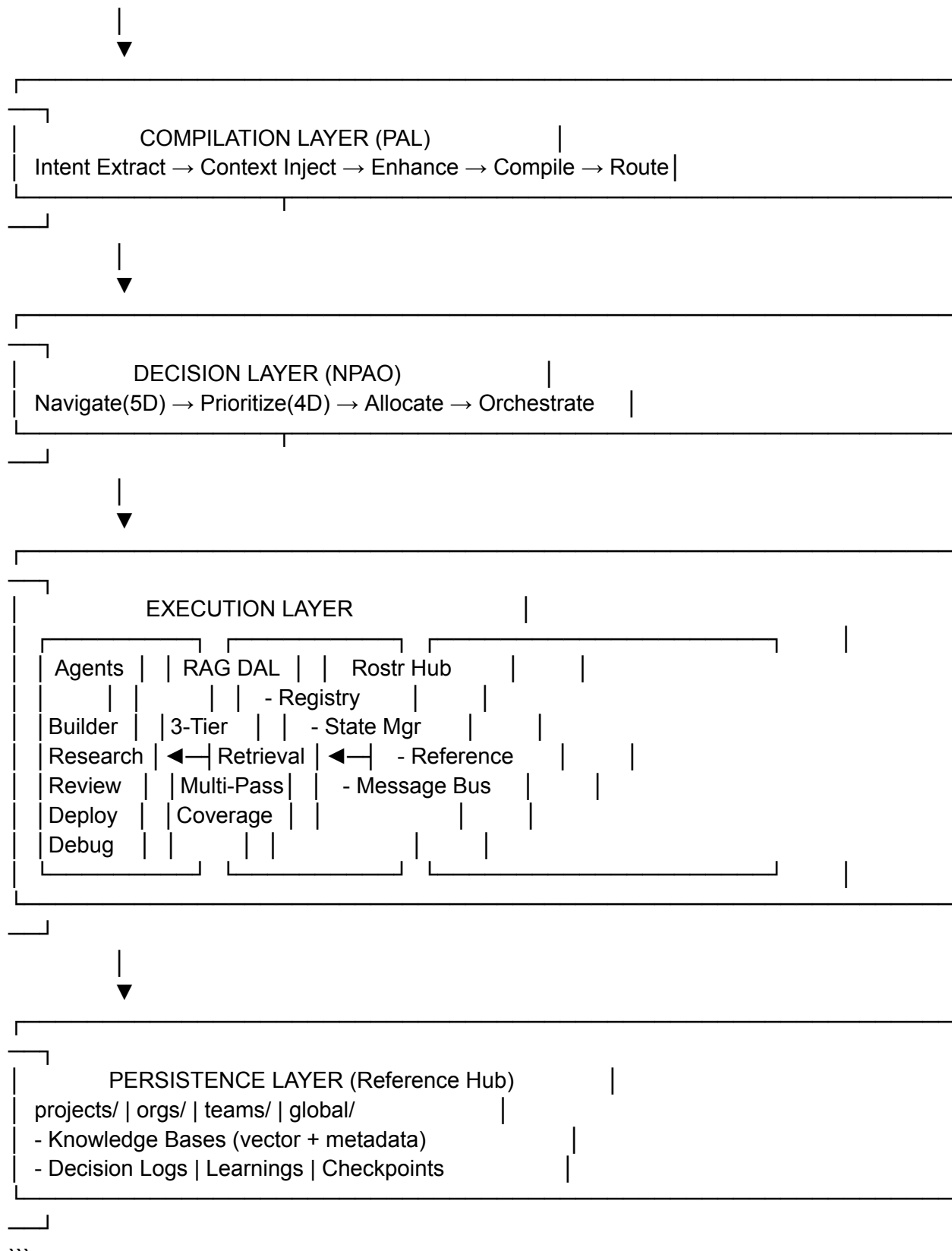
3. System Architecture

3.1 High-Level Design

ROSTR's architecture comprises four layers and four primary components operating on a shared persistent hub:

...





3.2 Information Flow

A task traverses the system as follows:

1. ****User Input**** — Natural language request, structured JTBD, or API call
2. ****PAL Compilation**** — Intent extracted, context injected from reference hub, semantically enhanced, compiled into agent manifest, routed to NPAO
3. ****NPAO Classification**** — Task classified into 5D phase, priority scored on 4 dimensions, allocated to capable agent based on phase/tools/load
4. ****Agent Execution**** — Agent receives compiled instruction, invokes tools, may call RAG DAL for knowledge needs, updates state in reference hub
5. ****RAG DAL (if triggered)**** — Multi-pass retrieval across tiered sources, coverage assessment, knowledge base ingestion, structured report to agent
6. ****State Persistence**** — Results, decisions, learnings written to reference hub namespace, available to all agents in scope
7. ****Output**** — Artifact, response, or action delivered to user; run logged

3.3 Component Interaction Invariants

****Invariant 1: PAL precedes execution**** — All agent invocations flow through PAL compilation (except explicit override mode). Ensures consistent instruction quality.

****Invariant 2: Phase classification precedes allocation**** — NPAO must navigate task to 5D phase before allocation. Ensures phase-appropriate agent selection.

****Invariant 3: Knowledge retrieval goes through RAG DAL**** — Agents do not perform ad-hoc web search. Centralized retrieval ensures credibility control and knowledge base population.

****Invariant 4: State updates persist to reference hub**** — Agents write learnings, decisions, artifacts to hub. Ensures knowledge compounding across sessions.

****Invariant 5: Cross-namespace access requires permission**** — Agents can read from attached namespaces (project/org/team) but require explicit permission for cross-project queries. Ensures scoped context.

These invariants enforce architectural coherence and prevent degenerate cases (agents bypassing compilation, performing unauthenticated retrieval, or operating statelessly).

4. PAL: Prompt Abstraction Layer

4.1 Design Rationale

The ****prompting bottleneck**** limits agent deployment: users must craft precise, well-structured prompts matching each agent's expectations. PAL reframes this as a ****compilation problem****: accept loosely-typed natural language intent; emit strictly-typed agent runtime manifests.

Analogy to software compilation:

Software Compiler	PAL Compiler
-----	-----
Source code (C, Python)	Natural language intent
Parsing & AST generation	Intent extraction
Type checking	Ambiguity resolution
Optimization passes	Semantic enhancement
Code generation	Runtime manifest compilation
Linker	Context injection from reference hub
Executable binary	Agent runtime config
Target architecture	Agent type (builder, researcher, deployer)

4.2 Five-Stage Compilation Pipeline

Stage 1: Intent Extraction

Input: Raw natural language string

Output: Structured intent object

```
```json
{
 "primary_intent": "what user wants to achieve (verb + object)",
 "domain": "code | design | research | ops | sales | content | deploy | debug",
 "subject": "thing being acted upon",
 "constraints": ["scope limits, method restrictions, output requirements"],
 "desired_output": "completion criteria",
 "urgency": "immediate | queued | scheduled",
 "ambiguity_score": 0.0-1.0
}
```
```

****Algorithm:****

1. Parse input for imperative verbs (build, research, fix, deploy, analyze)
2. Extract domain signals (keywords: "code", "design", "customer", "bug")
3. Identify constraints (explicit: "under 100 words", "use Python"; implicit: inferred from context)
4. Classify urgency (time signals: "now", "today", "next sprint")
5. Compute ambiguity score: $1.0 - (\text{explicit_parameters} / \text{total_required_parameters})$

****Example:****

Input: ``"We need a pricing page for our product"``

Extracted Intent:

```
```json
{
 "primary_intent": "create pricing page",
 "domain": "code + design",
 "subject": "pricing page",
 "constraints": ["for our product"],
 "desired_output": "deployable page",
 "urgency": "queued",
 "ambiguity_score": 0.7 // Missing: design specs, deployment target, content
}
```
```

Stage 2: Context Injection

Input: Intent object

Context Sources: Reference hub (project/org/team/session namespaces)

Output: Enriched intent with injected context

****Context Retrieval Algorithm:****

...

1. Load session state (current branch, recent changes, active files)
2. Load project context (architecture docs, conventions, CLAUDE.md)
3. Load org context (ICP, brand guidelines, approved messaging)
4. Load team context (shared conventions, tool preferences)
5. Embed intent.primary_intent → vector search reference hub
6. Retrieve top-k relevant prior decisions, learnings, artifacts
7. Inject into intent.context_payload

...

****Context Budget Management:****

Given a maximum token budget `T\_max` (e.g., 8000 tokens for Sonnet 4):

...

T_system = 1500 // System instructions, agent role

T_tools = 500 // Tool schemas

T_examples = 300 // Few-shot examples

T_context = T_max - T_system - T_tools - T_examples - T_output_reserve

...

Prioritize context injection:

1. Critical project state (current goals, active constraints)

2. Relevant prior decisions (vector similarity > 0.75)
3. Domain knowledge (ICP, brand, technical architecture)
4. Learnings from prior runs (if related to current intent)

Truncate lowest-priority context if budget exceeded.

Stage 3: Semantic Enhancement

Input: Context-enriched intent

Output: Precise, actionable instruction

****Enhancement Rules:****

1. ****Expand ambiguous verbs****
 - "improve" → "identify top 3 issues by severity, propose specific fix for each"
 - "check" → "verify X, Y, Z; flag anomalies; report status"
2. ****Add missing precision****
 - If intent lacks success criteria → inject: "done when: tests pass, code reviewed, deployed"
 - If output format unspecified → inject: "return as: markdown report | JSON | code file"
3. ****Decompose compound goals****
 - "Build and deploy pricing page" → [PreD: research competitor pricing] → [Design: wireframe + copy] → [Dev: implement] → [Deploy: ship]
4. ****Remove hedging****
 - "Maybe we should..." → "Do X"
 - "It would be nice if..." → "Implement Y"
5. ****Inject domain best practices****
 - For code tasks: "write tests", "follow repo conventions", "update docs"
 - For research tasks: "cite sources", "check multiple perspectives", "note confidence level"

****Example Enhancement:****

Before:

...

"Make the landing page better"

...

After:

...

Audit landing page against 5 conversion criteria:

1. Headline clarity: communicates value in <8 words

2. CTA placement: above fold, action verb, low friction
3. Social proof: testimonials, logos, or metrics visible
4. Load time: LCP < 2s
5. Mobile layout: tap targets ≥44px, readable text, smooth scroll

For each criterion:

- Score 0-10
- Describe what score=10 looks like
- Implement highest-impact fix
- Commit fix separately with message explaining change

Success criteria: All criteria ≥7/10, page loads <2s on 3G, mobile-first design verified.
...

Stage 4: Runtime Compilation

Input: Enhanced instruction

Output: Agent runtime manifest

```yaml

runtime:

agent\_type: builder | researcher | reviewer | designer | deployer | debugger  
model: claude-sonnet-4-6 | claude-opus-4 | gpt-4 | auto-select  
temperature: 0.0-1.0 // Higher for creative, lower for deterministic  
max\_parallel\_tasks: int  
timeout\_seconds: int

instructions:

system: "path/to/system\_prompt.md"  
behavior\_profile: "analytical | creative | operational | investigative"  
task\_description: "enhanced instruction from Stage 3"  
completion\_criteria: ["checklist"]  
escalation\_policy: "auto-proceed | require-approval | human-in-loop"

tools\_enabled:

allow: ["web\_search", "file\_system:read", "code\_execution", "api:hubspot"]  
deny: ["file\_system:write:production", "email:send"] // Safety guards

memory:

mode: "session | project | persistent"  
context\_sources: ["projects/acme", "orgs/acme-corp"]  
save\_triggers:

- decisions
- learnings

- artifacts

retrieve\_strategy: "vector\_similarity | recency | explicit\_query"

output:

format: "markdown | json | code | action"

destination: "return | file:path | api:endpoint"

verification: "none | test | human-review"

...

**\*\*Model Selection Heuristic:\*\***

```python

def select_model(intent: Intent, enhanced: str) -> str:

if intent.domain == "research" and len(enhanced) > 2000:

return "claude-opus-4" # Deep reasoning

elif intent.domain == "code" and intent.urgency == "immediate":

return "claude-sonnet-4-6" # Fast, high-quality

elif intent.domain == "design" or "creative" in enhanced.lower():

return "claude-opus-4" # Creative tasks

else:

return "claude-sonnet-4-6" # Default workhorse

...

Stage 5: Output Routing

Input: Runtime manifest

Output: Routed to appropriate execution layer

****Routing Logic:****

```python

def route(manifest: RuntimeManifest) -> ExecutionTarget:

if manifest.instructions.task\_description contains ["research", "analyze", "investigate"]:

return NPAO(phase="PreD", agent\_type="researcher", trigger\_ragdal=True)

elif manifest.instructions.task\_description contains ["design", "wireframe", "UI"]:

return NPAO(phase="Design", agent\_type="designer")

elif manifest.instructions.task\_description contains ["build", "implement", "code"]:

return NPAO(phase="Development", agent\_type="builder")

elif manifest.instructions.task\_description contains ["deploy", "ship", "release"]:

return NPAO(phase="Deployment", agent\_type="deployer")

elif manifest.instructions.task\_description contains ["bug", "fix", "error", "broken"]:

return NPAO(phase="Debugging", agent\_type="debugger", priority\_override=True)

```

else:
 # Ambiguous - use LLM for classification
 return classify_with_llm(manifest)
...

```

### ### 4.3 Technical Specifications

| Property                 | Value                                                   |
|--------------------------|---------------------------------------------------------|
| Latency overhead         | 200-400ms (Haiku), 400-800ms (Sonnet)                   |
| Cost per compilation     | ~\$0.001 (Haiku), ~\$0.003 (Sonnet)                     |
| Context window usage     | 500-1,500 tokens                                        |
| Supported input formats  | Text, voice transcript, structured JSON                 |
| Supported output formats | Agent manifest (YAML/JSON), skill file, API payload     |
| Enhancement model        | claude-haiku-4-5 (default), claude-sonnet-4-6 (complex) |
| Storage                  | Stateless by default; session cache optional            |

### ### 4.4 Evaluation Metrics (Proposed)

**Hypothesis 1:** PAL-enhanced instructions yield higher task completion rates than baseline prompts.

**Experimental Design:**

- A/B test: 100 tasks, 50 baseline (user-written prompts), 50 PAL-enhanced
- Measure: Completion rate, iterations to completion, user satisfaction
- Control variables: Task complexity, agent capability, domain

**Hypothesis 2:** Context injection reduces token usage while maintaining quality.

**Experimental Design:**

- Compare: (A) full context dump, (B) PAL context budgeting, (C) no context
- Measure: Token usage, task completion quality, context relevance score
- Tasks: 50 multi-session workflows where context continuity matters

**Hypothesis 3:** Semantic enhancement improves agent instruction clarity.

**Experimental Design:**

- Human evaluation: 50 enhanced instructions vs. 50 baseline
- Rubric: Specificity (1-5), actionability (1-5), completeness (1-5)
- Blind reviewers rate both sets

---

## ## 5. RAG DAL: Dynamic Acquisition Layer

### ### 5.1 Design Rationale

Standard RAG suffers from: (1) **shallow retrieval**—single-pass search without iterative refinement; (2) **source quality blindness**—treating Reddit and Nature equally; (3) **no self-awareness**—system doesn't know what it doesn't know, can't detect coverage gaps.

RAG DAL introduces: (1) **three-tier source credibility hierarchy**; (2) **autonomous multi-pass loop with convergence criteria**; (3) **gap detection and targeted re-search**; (4) **persistent knowledge base with provenance**.

### ### 5.2 Three-Tier Source Architecture

Sources stratified by credibility:

**Tier 1: Primary & Authoritative** (credibility = 1.0)

- Academic databases: arXiv, PubMed, JSTOR, Google Scholar
- Encyclopedias: Wikipedia (citations only), Britannica
- Official docs: .gov, university repositories, standards bodies
- **Usage:** Always attempt first; establish ground truth

**Tier 2: Verified & Editorial** (credibility = 0.75)

- Major news: Reuters, AP, BBC, NYT, WSJ
- Trade publications: TechCrunch, IEEE Spectrum, domain journals
- Peer-reviewed with DOI
- Government portals (.gov, .edu)
- Analyst reports: Gartner, McKinsey, Forrester
- **Usage:** Contextualize Tier 1; current events; industry trends

**Tier 3: Community & UGC** (credibility = 0.40)

- Blogs, Substack, personal sites
- Social: LinkedIn, Twitter/X, Reddit
- Forums: Stack Overflow, Hacker News
- User reviews: G2, Trustpilot
- Podcast/video transcripts
- **Usage:** Real-world signal, sentiment, challenge/validate Tier 1-2

**Rationale:** Not all sources are equal. Academic papers > blog posts for factual claims. UGC valuable for sentiment, adoption patterns, real-world edge cases. Explicit credibility weighting prevents contamination of high-confidence findings with low-quality sources.

### ### 5.3 Autonomous Multi-Pass Retrieval Loop

**\*\*Algorithm:\*\***

...

INPUT: query Q, confidence\_threshold  $\theta$  (default 0.8)

OUTPUT: structured report R, knowledge base entries K

// Pass 1: Broad Sweep

searches\_1 = generate\_search\_queries(Q, count=5, strategy="broad")

results\_1 = execute\_searches(searches\_1, tiers=[1,2,3])

corpus\_1 = extract\_content(results\_1)

sub\_topics = decompose\_query(Q)

confidence\_scores = {}

FOR EACH topic IN sub\_topics:

    confidence\_scores[topic] = assess\_coverage(topic, corpus\_1)

    // Coverage based on: source count, tier distribution, consistency

IF all(confidence\_scores.values  $\geq \theta$ ):

    RETURN generate\_report(corpus\_1, confidence\_scores)

// Pass 2: Gap Fill

gaps = [topic for topic, score in confidence\_scores.items() if score  $< \theta$ ]

searches\_2 = generate\_targeted\_searches(gaps, count=2 per gap)

results\_2 = execute\_searches(searches\_2, tiers=[1,2])

corpus\_2 = corpus\_1 + extract\_content(results\_2)

FOR EACH topic IN gaps:

    confidence\_scores[topic] = assess\_coverage(topic, corpus\_2)

IF all(confidence\_scores.values  $\geq \theta$ ):

    RETURN generate\_report(corpus\_2, confidence\_scores)

// Pass 3: Deep Verification

still\_low = [topic for topic, score in confidence\_scores.items() if score  $< \theta - 0.1$ ]

searches\_3 = generate\_primary\_source\_searches(still\_low)

results\_3 = execute\_searches(searches\_3, tiers=[1]) // Tier 1 only

corpus\_3 = corpus\_2 + extract\_content(results\_3)

FOR EACH topic IN still\_low:

    confidence\_scores[topic] = assess\_coverage(topic, corpus\_3)

    IF confidence\_scores[topic]  $< 0.7$ :

        mark\_as\_uncertain(topic)

RETURN generate\_report(corpus\_3, confidence\_scores, uncertainties)

```
// Optional Pass 4 (triggered if ≥ 2 topics < 0.6)
IF count(score < 0.6 for score in confidence_scores.values) >= 2:
 searches_4 = generate_deep_searches(very_low_topics, count=3)
 results_4 = execute_searches(searches_4, tiers=[1,2,3])
 corpus_4 = corpus_3 + extract_content(results_4)
 final_scores = assess_coverage(all_topics, corpus_4)
 mark_unresolvable(topics where final_scores < 0.6)
 RETURN generate_report(corpus_4, final_scores, uncertainties, unresolvable)
...
```

#### ### 5.4 Coverage Assessment Criteria

A sub-topic is "covered" (confidence  $\geq 0.8$ ) when:

1. **Source Count:**  $\geq 2$  Tier 1 or Tier 2 sources confirm the core claim
2. **No Contradictions:** High-credibility sources don't contradict each other on key facts
3. **Recency:** Information <90 days old (for time-sensitive topics) OR verified timeless (for foundational knowledge)
4. **Completeness:** All extracted sub-questions from original query have answers

**Confidence Scoring Formula:**

```
...
confidence(topic) = w_sources * source_score(topic) +
 w_consistency * consistency_score(topic) +
 w_tier * tier_distribution_score(topic) +
 w_recency * recency_score(topic)
```

where:

```
w_sources = 0.35
w_consistency = 0.30
w_tier = 0.25
w_recency = 0.10
```

```
source_score = min(1.0, confirmed_sources / 2) // 2+ sources → 1.0
consistency_score = 1.0 - (contradictions / total_claims)
tier_distribution_score = (Tier1_count * 1.0 + Tier2_count * 0.75 + Tier3_count * 0.4) /
total_sources
recency_score = 1.0 if (current_date - published_date < 90 days) OR timeless else 0.5
...
```

#### ### 5.5 Knowledge Base Ingestion

Retrieved content transformed into structured entries:

```

```json
{
  "entry_id": "uuid",
  "query_origin": "original question triggering retrieval",
  "content": "extracted main text (cleaned, normalized)",
  "summary": "3-5 sentence distillation",
  "source": {
    "url": "https://...",
    "title": "page/paper title",
    "author": "person or org",
    "published_date": "ISO 8601",
    "retrieved_date": "ISO 8601",
    "tier": "1 | 2 | 3",
    "credibility_score": "0.0-1.0"
  },
  "metadata": {
    "topics": ["tags"],
    "entities": ["named entities: people, orgs, places"],
    "data_type": "factual | opinion | statistical | procedural"
  },
  "vector_embedding": [float array],
  "confidence": "0.0-1.0, // From coverage assessment",
  "verification_status": "verified | uncertain | open"
}
```

```

**\*\*Ingestion Pipeline:\*\***

```

...
Web Page/PDF → Format Detection → Content Extraction
→ Normalization (Unicode, dedupe, language detection)
→ Metadata Tagging (source, date, tier, credibility)
→ Chunk & Embed (512 tokens, 64 overlap, text-embedding-3-small)
→ Store in vector DB (pgvector/Pinecone/Weaviate)
→ Index by namespace (project/org/team/global)
...

```

#### ### 5.6 Multi-Namespace Knowledge Architecture

Knowledge base organized by access scope:

```

...
knowledge_base/
└── project/{project_id}/ // Project-specific research

```

```

├── org/{org_id}/ // Organization-wide knowledge
├── team/{team_id}/ // Team-scoped findings
├── domain/{topic}/ // Domain expertise (marketing, ML, etc.)
└── global/ // Public shared knowledge
...

```

#### **\*\*Access Control:\*\***

- Agents automatically query namespaces they're attached to
- Cross-namespace queries require explicit permission
- Prevents context pollution (personal project research doesn't leak to org-wide queries)

### ### 5.7 Technical Specifications

```

Property	Value
Search execution	Parallel across tiers (non-blocking)
Max passes per query	4 (configurable)
Confidence threshold	0.8 (configurable: 0.6-0.9)
Chunk size	512 tokens, 64-token overlap
Embedding model	text-embedding-3-small (OpenAI) or equivalent
Vector database	Supabase pgvector / Pinecone / Weaviate
Cache TTL	72 hours (configurable by namespace)
Max sources per query	25 (configurable)
Retrieval latency	30-90 seconds per full pipeline
Storage overhead	~2KB per entry + embedding (1536 dims × 4 bytes = 6KB)

```

### ### 5.8 Output Format

RAG DAL returns structured markdown reports:

```
```markdown
```

```
# RAG DAL Report: [Query]
```

```
**Passes run:** 3
```

```
**Overall confidence:** 8.2/10
```

```
**Sources consulted:** 18
```

```
**Date:** 2026-04-12T14:32:00Z
```

Key Findings

[Sub-topic 1]

```
**Answer:** [Synthesis of findings]
```

```
**Confidence:** 9.1/10
```

```
**Primary sources:**
```

- [Title](URL) — Tier 1, credibility 0.95, published 2026-01-15
- [Title](URL) — Tier 1, credibility 1.0, published 2025-12-03

****Supporting sources:****

- [Title](URL) — Tier 2, credibility 0.78, published 2026-02-20

[Sub-topic 2]

...

Open Questions (confidence < 7/10)

- ****[Question]:**** What we found, why uncertain (e.g., "Only 1 Tier 1 source, contradicted by Tier 2 source")

Gap Indicators

- ****[Topic not covered]:**** Reason (no sources found / too recent / contradicting claims)

Source Index

#	Title	URL	Tier	Published	Credibility
1	...	...	1	2026-01-15	0.95
2	...	...	2	2026-02-10	0.75
...					

5.9 Evaluation Metrics (Proposed)

****Hypothesis 1:**** Multi-pass retrieval achieves higher coverage than single-pass.

****Experimental Design:****

- 50 complex queries requiring multi-faceted answers
- Baseline: Single-pass RAG (retrieve top-10, rank, augment)
- RAG DAL: Multi-pass with convergence
- Measure: Coverage (% sub-questions answered), precision (% correct claims), source quality (avg tier)
- Human evaluation: Domain experts rate completeness and accuracy

****Hypothesis 2:**** Tier stratification improves output reliability.

****Experimental Design:****

- 100 factual queries with known ground truth
- Condition A: No tier weighting (all sources equal)

- Condition B: RAG DAL tier weighting (1.0, 0.75, 0.40)
- Measure: Factual accuracy, hallucination rate, citation quality
- Control: Same search results, only weighting differs

Hypothesis 3: Gap detection reduces unanswered queries.

Experimental Design:

- 50 queries with intentional gaps (sub-topics not covered in first pass)
- Baseline: Fixed 2-pass retrieval
- RAG DAL: Gap-triggered targeted re-search
- Measure: % queries with confidence ≥ 0.8 per sub-topic, retrieval efficiency (passes needed)

Hypothesis 4: Knowledge persistence reduces redundant retrieval.

Experimental Design:

- 20 agents working on related tasks over 2 weeks
- Condition A: No shared KB (each agent re-retrieves)
- Condition B: RAG DAL persistent KB with cross-agent access
- Measure: Retrieval API calls, cost, latency, context reuse rate

6. NPAO: Navigate, Prioritize, Allocate, Orchestrate

6.1 Design Rationale

Multi-agent systems fail when: (1) **tasks routed naively** (keyword "test" → QA agent, regardless of whether it's exploratory testing in PreD or regression testing pre-deployment); (2) **no priority model** (FIFO execution regardless of business impact or dependencies); (3) **stateless allocation** (agent with most context about task not preferentially selected).

NPAO introduces: (1) **5D phase taxonomy** structuring workflow lifecycle; (2) **multi-dimensional priority scoring**; (3) **context-aware agent allocation**.

6.2 The 5D Phase Taxonomy

All work classified into five phases:

Phase 0: PreD (Pre-Development / Drafting)

- **Purpose:** Determine *if* to build before deciding *how*
- **Activities:** Problem definition, competitive research (RAG DAL heavy), assumption identification, feasibility assessment, go/no-go decision
- **Critical Question:** "Is this worth building?"

- **Completion Criteria:**
 - ☐ Problem stated in one sentence
 - ☐ Target user identified with specificity
 - ☐ ≥ 3 alternatives considered and rejected with reasons
 - ☐ Success criteria defined (measurable outcomes)
 - ☐ Known unknowns documented
 - ☐ Decision: build now / build later / don't build
- **Output Artifact:** PreD Report (go/no-go document, not a spec)
- **Agent Behavior:** Research-dominant, RAG DAL calls, no code written, no designs committed

Phase 1: Design

- **Purpose:** Define *what* to build and *how* it should behave
- **Activities:** Architecture design, UI/UX wireframes, data model definition, API contracts, tech stack decisions
- **Critical Question:** "What exactly are we building?"
- **Completion Criteria:**
 - ☐ Architecture diagram exists
 - ☐ User flows documented
 - ☐ Data models defined
 - ☐ Interfaces (API, UI) specified
 - ☐ Technology choices made with rationale
 - ☐ Edge cases identified
- **Output Artifact:** Design Spec (blueprints, not code)
- **Agent Behavior:** Design agents, architecture reviewers, PAL compiles design prompts

Phase 2: Development

- **Purpose:** Build it
- **Activities:** Feature implementation, testing, code review, documentation, refactoring
- **Critical Question:** "Does it work?"
- **Completion Criteria:**
 - ☐ All specified features implemented
 - ☐ Test coverage $\geq$ threshold (configurable, e.g., 80%)
 - ☐ Code review passed
 - ☐ No known blocking bugs
 - ☐ Documentation updated
- **Output Artifact:** Working, tested code
- **Agent Behavior:** Builder agents dominant, reviewer agents post-change, PAL compiles code tasks

****Phase 3: Deployment****

- ****Purpose:**** Ship it safely
- ****Activities:**** CI/CD execution, staging verification, production deploy, monitoring setup, rollback planning
- ****Critical Question:**** "Is it safe to ship? Is it working after ship?"
- ****Completion Criteria:****
 - ☐ Staging QA passed
 - ☐ Performance benchmarks within targets
 - ☐ Security audit passed (if required)
 - ☐ Monitoring and alerting active
 - ☐ Rollback procedure documented and tested
 - ☐ Production deploy verified (canary checks)
- ****Output Artifact:**** Live, monitored feature
- ****Agent Behavior:**** Ship agents, canary agents, benchmark agents

****Phase 4: Debugging****

- ****Purpose:**** Fix what's broken
- ****Activities:**** Bug reproduction, root cause analysis, fix implementation, regression testing, post-mortem (if significant)
- ****Critical Question:**** "What broke, why, how do we prevent it again?"
- ****Completion Criteria:****
 - ☐ Bug reproduced reliably
 - ☐ Root cause identified (not just symptom patched)
 - ☐ Fix implemented and tested
 - ☐ Regression test added
 - ☐ Post-mortem written (if P0/P1)
- ****Output Artifact:**** Documented fix + prevention measure
- ****Agent Behavior:**** Investigate agents (root cause before fix), PAL compiles investigation prompts

****Phase Transitions:****

- Sequential: PreD → Design → Development → Deployment
- Debugging can interrupt any phase (priority override)
- Failed PreD → Archive (don't build)
- Debugging may reveal design flaw → return to Design for that component

6.3 Multi-Dimensional Priority Scoring

Each task scored on four dimensions:

****Dimension 1: Phase Urgency (0-10)****

Phase	Base Score	Modifiers
Debugging (P0 production outage)	10	None — always top priority
Deployment (active release)	8	+2 if revenue-impacting
Development (blocked sprint)	6	+2 if dependency blocker
Design (pre-development)	4	+1 if team waiting
PreD (research)	2	+3 if strategic deadline imminent

****Dimension 2: Dependency Impact (0-10)****

Count of tasks blocked by this one:

- 0 tasks blocked → 0
- 1-2 tasks blocked → 3
- 3-5 tasks blocked → 6
- 6+ tasks blocked → 10

****Dimension 3: Business Impact (0-10)****

- Revenue directly affected (payment broken, customer-facing bug) → 9-10
- User experience affected (feature broken, performance degraded) → 6-8
- Team productivity affected (tooling broken, workflow blocked) → 4-6
- Internal tooling only (developer convenience) → 2-4
- Nice to have (low user impact) → 0-2

****Dimension 4: Resource Efficiency (0-10)****

Value per agent-hour:

- <1 hour, high confidence of success → 10
- 1-4 hours, moderate complexity → 7
- 4-8 hours, high complexity → 4
- Multi-day, research-heavy, uncertain → 2

****Composite Priority Score:****

...

$$\begin{aligned} \text{Priority} = & (\text{Phase_Urgency} \times 0.35) + \\ & (\text{Dependency_Impact} \times 0.30) + \\ & (\text{Business_Impact} \times 0.25) + \\ & (\text{Resource_Efficiency} \times 0.10) \end{aligned}$$

Result: 0.0-10.0

Thresholds:

- ≥ 7.0: Immediate allocation
- 4.0-6.9: Queued for next available agent
- < 4.0: Async processing or backlog

...

****Example:****

Task: "Fix checkout payment bug (production)"

- Phase: Debugging (P0) → Urgency = 10
- Dependency: 0 other tasks blocked → Impact = 0
- Business: Revenue directly affected → Impact = 10
- Resource: Estimated 2 hours, moderate complexity → Efficiency = 7

$$\begin{aligned}\text{Priority} &= (10 \times 0.35) + (0 \times 0.30) + (10 \times 0.25) + (7 \times 0.10) \\ &= 3.5 + 0 + 2.5 + 0.7 = 6.7\end{aligned}$$

Interpretation: High priority (≥6.0), allocate to debug agent immediately, but not quite P0 level (would need dependency blocker or higher efficiency).

6.4 Agent Allocation Algorithm

Once tasks prioritized, allocate to agents:

```
```python
def allocate_task(task: Task, agents: List[Agent]) -> Agent:
 """
 Allocate task to best-suited available agent.
 """
 # 1. Filter to eligible agents
 eligible = []
 for agent in agents:
 if not task.phase in agent.phases:
 continue # Agent not specialized for this phase
 if not all(tool in agent.tools for tool in task.required_tools):
 continue # Agent lacks required tools
 if agent.current_tasks >= agent.max_parallel_tasks:
 continue # Agent at capacity
 if task.dependencies_unresolved():
 continue # Can't start yet

 eligible.append(agent)

 if not eligible:
```

```

 return None # Queue for later

2. Score each eligible agent
scores = {}
for agent in eligible:
 context_score = compute_context_score(agent, task)
 specialization_score = compute_specialization_score(agent, task)
 load_score = 1.0 - (agent.current_tasks / agent.max_parallel_tasks)

 scores[agent] = (
 context_score * 0.50 + # Prefer agent with task context
 specialization_score * 0.35 + # Prefer specialist
 load_score * 0.15 # Slight preference for less loaded
)

3. Allocate to highest-scoring agent
best_agent = max(scores, key=scores.get)
assign_task(best_agent, task)
return best_agent

def compute_context_score(agent: Agent, task: Task) -> float:
 """
 How much does this agent already know about the task context?
 """
 # Check agent's memory for related prior work
 related_tasks = agent.memory.query(task.subject, limit=5)
 if not related_tasks:
 return 0.0

 # Vector similarity of current task to agent's prior work
 similarities = [cosine_sim(task.embedding, rt.embedding) for rt in related_tasks]
 return max(similarities)

def compute_specialization_score(agent: Agent, task: Task) -> float:
 """
 How well does agent's skill set match task requirements?
 """
 required_capabilities = task.required_capabilities
 agent_capabilities = set(agent.capabilities)

 if not required_capabilities:
 return 0.5 # No specific requirements, all agents equal

```

```

 overlap = len(required_capabilities & agent_capabilities)
 return overlap / len(required_capabilities)
...

```

### ### 6.5 Orchestration Patterns

NPAO supports four execution patterns:

#### \*\*1. Sequential Chain\*\*

...

Task A → Task B → Task C

...

Used when output of A is input to B. Example: PreD report → Design spec → Development.

#### \*\*2. Parallel Fan-Out\*\*

...

```

 ┌─ Task B
Task A → ┤─ Task C
 └─ Task D
...

```

Used when tasks are independent. Example: Multiple feature implementations in parallel.

#### \*\*3. Aggregation Fan-In\*\*

...

```

Task A ┌
Task B ┤───→ Task E (synthesizes A, B, C)
Task C └
...

```

Used when results must be combined. Example: Multiple RAG DAL queries feeding into single research report.

#### \*\*4. Conditional Branch\*\*

...

```

Task A → [decision point] → Task B (if condition true)
 → Task C (if condition false)
...

```

Used when next step depends on outcome. Example: PreD go/no-go branching to Design or Archive.

### ### 6.6 Technical Specifications

```

| Property | Value |
|-----|-----|

```

Priority re-computation frequency	On task completion, state change, or explicit trigger
Agent capacity model	Max parallel tasks (configurable per agent, default 3)
Context scoring method	Vector similarity (task embedding vs. agent memory)
Allocation latency	<100ms for typical load (50 tasks, 10 agents)
Phase classification	Keyword matching + LLM fallback for ambiguous cases

### ### 6.7 Evaluation Metrics (Proposed)

**\*\*Hypothesis 1:\*\*** Multi-dimensional priority scoring outperforms FIFO or single-dimension scoring.

**\*\*Experimental Design:\*\***

- Simulated workload: 100 tasks with varied phase, dependencies, business impact
- Conditions: (A) FIFO, (B) Phase-only priority, (C) NPAO 4D scoring
- Measure: Time to complete high-business-impact tasks, dependency violation rate, resource utilization
- Success criteria: NPAO completes P0/P1 tasks 30% faster, fewer dependency violations

**\*\*Hypothesis 2:\*\*** Phase-aware allocation improves task success rate.

**\*\*Experimental Design:\*\***

- 50 tasks across all 5D phases
- Condition A: Random agent allocation
- Condition B: Phase-aware allocation (agent must have phase in specialization)
- Measure: First-attempt success rate, iterations to completion, human intervention rate

**\*\*Hypothesis 3:\*\*** Context-aware allocation reduces task setup time.

**\*\*Experimental Design:\*\***

- 30 multi-session tasks requiring context continuity
- Condition A: Round-robin allocation (ignore context)
- Condition B: NPAO context scoring (prefer agent with prior task knowledge)
- Measure: Time spent on context gathering/clarification, task completion latency

**\*\*Hypothesis 4:\*\*** PreD phase reduces wasted development effort.

**\*\*Experimental Design:\*\***

- 20 projects: 10 with PreD phase enforced, 10 without (skip to Design)
- Track: Projects canceled mid-development, scope changes post-Design, re-work hours
- Hypothesis: PreD group has lower cancelation rate, fewer scope changes, less re-work

---

## ## 7. Rostr Hub: Agent Operating System

### ### 7.1 Design Rationale

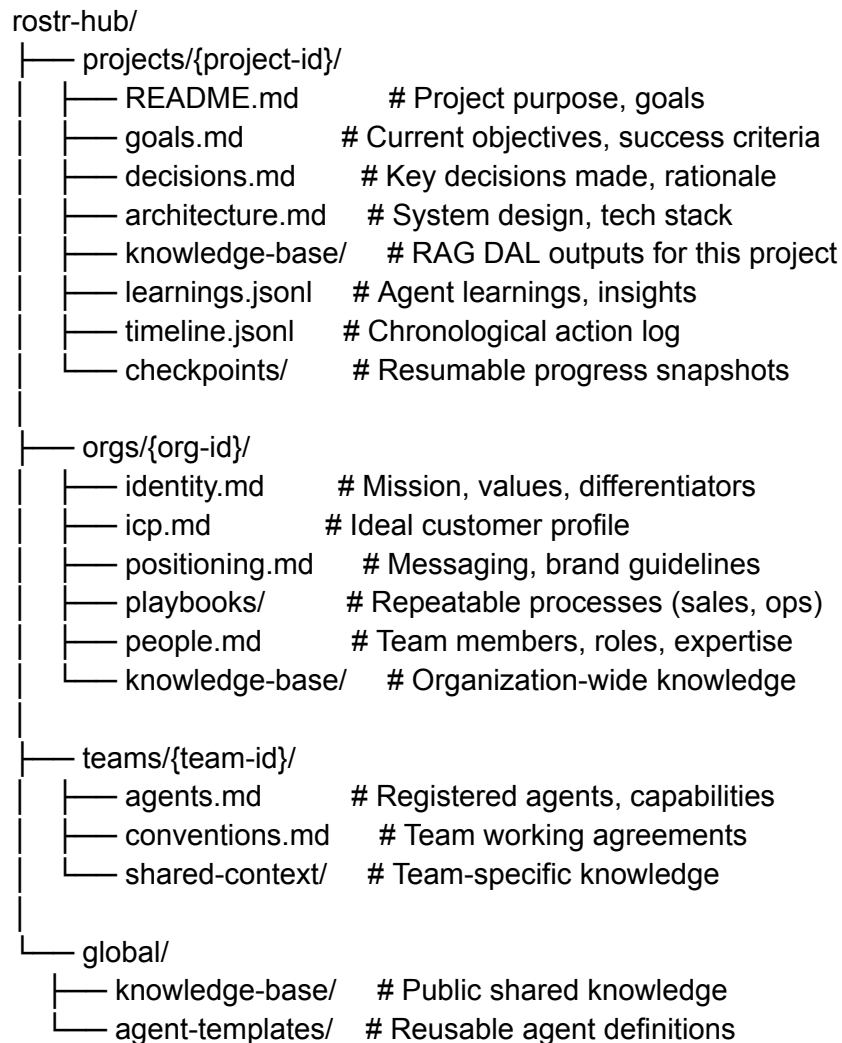
Agent teams need: (1) **persistent state** across sessions; (2) **shared knowledge** across agents; (3) **agent registry** for discovery and capability matching; (4) **communication protocols** for coordination.

Rostr Hub provides this infrastructure as a central platform all agents attach to.

### ### 7.2 Reference Hub Architecture

Multi-namespace knowledge hierarchy:

...



...

**\*\*Access Patterns:\*\***

- **Read:** Any agent can read from attached namespaces (project, org, team, global)
- **Write:** Agents write learnings, decisions, artifacts after task completion
- **Cross-namespace:** Agents can query other projects/orgs with explicit permission

**Example:**

Builder Agent working on `projects/acme-product`:

- Auto-loads: `projects/acme-product/`, `orgs/acme-corp/`, `global/`
- Context: Current goals, recent decisions, org ICP, prior implementations
- On completion: Writes code to repo, logs decision to `decisions.md`, saves learnings to `learnings.jsonl`

Next session (same or different agent):

- Loads same namespaces
- Sees prior agent's decisions and learnings
- No context loss

### ### 7.3 Agent Registration Schema

Agents declare capabilities on registration:

```
```json
{
  "agent_id": "uuid-v4",
  "name": "Builder Agent — Feature Implementation",
  "description": "Implements features, writes tests, edits code",
  "type": "builder", // Enum: builder | researcher | reviewer | designer | deployer | debugger |
orchestrator
  "capabilities": [
    "code_generation",
    "file_editing",
    "api_integration",
    "test_writing",
    "refactoring"
  ],
  "tools": [
    "file_system:read",
    "file_system:write",
    "code_execution",
    "bash",
    "web_search"
  ],
  "phases": ["development", "debugging"], // Which 5D phases this agent handles
}
```

```

"model": "claude-sonnet-4-6",
"context_requirements": [
  "projects/{id}/architecture.md",
  "projects/{id}/conventions.md"
],
"output_formats": ["code", "diff", "markdown"],
"max_parallel_tasks": 3,
"contact": {
  "protocol": "claude_code_skill | api | mcp",
  "endpoint": "/path/to/skill.md or https://api.endpoint"
},
"performance_stats": {
  "tasks_completed": 127,
  "avg_completion_time_minutes": 18,
  "success_rate": 0.94
}
}
...

```

****Registry Operations:****

- `register(agent\_spec)` — Add agent to registry
- `discover(phase, capabilities, available=True)` — Find agents matching criteria
- `update\_stats(agent\_id, task\_result)` — Track performance
- `deregister(agent\_id)` — Remove agent

7.4 Four-Level State Management

****Level 1: Session State** (ephemeral)**

- Active tasks, in-progress work, current context window
- Cleared when session ends
- Storage: In-memory or Redis cache

****Level 2: Project State** (persistent)**

- Decisions, artifacts, learnings, history, current status
- Persists indefinitely
- Storage: File-based (markdown, JSONL) + vector DB for searchability

****Level 3: Organization State** (evolving)**

- Identity, ICP, positioning, team structure, playbooks
- Updated periodically as org evolves
- Storage: File-based, version-controlled

****Level 4: Agent State** (portable)**

- Agent skills, preferences, calibration data, performance history

- Portable across projects
- Storage: Agent-specific namespace, can be exported/imported

****State Persistence Protocol:****

...

After significant agent action:

1. Agent generates state update object:

```
{
  "type": "decision | learning | artifact | milestone",
  "content": "...",
  "namespace": "projects/acme",
  "tags": ["feature-x", "payment-integration"]
}
```

2. Route to appropriate namespace in Reference Hub
3. If type == "learning" → append to learnings.jsonl
4. If type == "decision" → append to decisions.md
5. If type == "artifact" → save to project files
6. If type == "milestone" → create checkpoint snapshot
7. Log to timeline.jsonl with timestamp, agent_id, action
8. Update vector DB for searchability

...

7.5 Communication Protocols

Agents communicate through three channels:

****1. Synchronous Task Assignment****

Orchestrator → Agent direct task handoff:

```
```json
{
 "protocol": "sync",
 "from": "orchestrator",
 "to": "builder-agent-01",
 "task": {
 "id": "task-uuid",
 "type": "implementation",
 "phase": "development",
 "instruction": "Implement user authentication using Supabase Auth",
 "context": {
 "project_id": "acme-product",
 "priority": 7.2,
 "dependencies": ["design-approved"],

```

```

 "related_tasks": ["task-uuid-2"]
 },
 "deadline": "2026-04-15T18:00:00Z",
 "on_complete": "notify_orchestrator"
}
}
...

```

Agent processes task, returns result, orchestrator marks complete.

## **\*\*2. Asynchronous Message Bus\*\* (Pub/Sub)**

For non-blocking notifications:

```

```json
{
  "protocol": "async",
  "topic": "projects/acme/deployments",
  "event": "deployment_complete",
  "payload": {
    "environment": "production",
    "version": "v1.2.3",
    "status": "success",
    "url": "https://acme-product.com"
  },
  "subscribers": ["canary-agent", "retro-agent", "orchestrator"]
}
...

```

Subscribers receive notification, can react (e.g., canary agent starts health monitoring).

****3. Knowledge Queries****

Agent queries Reference Hub directly:

```

```json
{
 "protocol": "query",
 "from": "builder-agent-01",
 "namespace": "projects/acme/knowledge-base",
 "query": "How do we integrate Stripe for subscription payments?",
 "max_results": 5,
 "filters": {
 "tags": ["payment", "stripe"],

```

```

 "date_range": "last_90_days"
 }
}
...

```

Hub returns relevant entries (vector search + metadata filtering).

#### ### 7.6 Technical Stack

Component	Technology	Rationale
Hub core	FastAPI (Python)	High performance, async support, OpenAPI auto-docs
State storage	Supabase (PostgreSQL + pgvector)	Relational + vector in one, managed service
Message bus	Redis Pub/Sub	Fast, low-latency, simple
Vector search	pgvector (Supabase) or Pinecone	Embedding similarity for knowledge retrieval
Dashboard	Next.js 15	React-based, server components, fast
Agent SDK	Python + TypeScript	Most common agent languages
Protocols	REST + WebSocket + MCP	REST for sync, WebSocket for streaming, MCP for modular tools
Authentication	Supabase Auth or Microsoft Entra	Managed identity, SSO support
Observability	OpenTelemetry + Application Insights	Distributed tracing, cost tracking

#### ### 7.7 Evaluation Metrics (Proposed)

**Hypothesis 1:** Persistent context reduces task setup time.

**Experimental Design:**

- 30 multi-session tasks
- Condition A: Stateless (agents start from scratch each session)
- Condition B: Rostr Hub (agents load project state)
- Measure: Time to first meaningful action, context-gathering questions, user re-briefing count

**Hypothesis 2:** Cross-agent knowledge sharing improves efficiency.

**Experimental Design:**

- 5 agents working on related sub-tasks in same project
- Condition A: Isolated (each agent's findings private)
- Condition B: Shared knowledge base (all agents read from project KB)
- Measure: Duplicate research instances, time to task completion, knowledge reuse rate

**Hypothesis 3:** Agent allocation using registry improves match quality.

## **\*\*Experimental Design:\*\***

- 50 tasks with varied requirements
- Condition A: Manual agent selection
- Condition B: Registry-based allocation (capability + phase matching)
- Measure: Task success rate, iterations to completion, capability mismatch rate

---

## **## 8. System Integration and Reference Implementation**

### **### 8.1 Integration Flow**

Full task lifecycle through integrated system:

...

User: "We need to add Stripe subscription billing to our product"

#### **1. PAL Compilation**

- Intent: "add subscription billing"
- Domain: code + integration
- Constraints: "Stripe", "subscription"
- Ambiguity: 0.6 (missing: billing tiers, trial period, proration)
- Enhancement: "Research Stripe subscription API. Design billing tiers. Implement subscription flow with Stripe Checkout. Test full billing cycle. Deploy with monitoring."
- Routing: Multi-phase task → NPAO

#### **2. NPAO Phase Decomposition**

Phase 0 (PreD):

- Task 1.0: Research Stripe Subscription API [Research Agent → RAG DAL]
- Task 1.1: Analyze competitor billing models [Research Agent → RAG DAL]
- Task 1.2: Define billing tiers (Starter, Pro, Enterprise) [Planning Agent]

Phase 1 (Design):

- Task 2.0: Design subscription schema (users, subscriptions, plans) [Architect Agent]
- Task 2.1: Design billing flow UX [Design Agent]
- Task 2.2: API contract for subscription endpoints [Architect Agent]

Phase 2 (Development):

- Task 3.0: Implement Stripe webhook handlers [Builder Agent]
- Task 3.1: Build subscription management UI [Builder Agent]
- Task 3.2: Write integration tests [Builder Agent]
- Task 3.3: Code review [Review Agent]

Phase 3 (Deployment):

- Task 4.0: Deploy to staging, test full flow [Deploy Agent]
- Task 4.1: Production deploy with canary [Deploy Agent]
- Task 4.2: Monitor webhook delivery [Canary Agent]

### 3. Prioritization

- Phase urgency: PreD = 2 (base)
- Dependency impact: 0 (greenfield)
- Business impact: 9 (revenue-generating feature)
- Resource efficiency: 4 (multi-day project)
- Priority =  $(2 \times 0.35) + (0 \times 0.30) + (9 \times 0.25) + (4 \times 0.10) = 3.35$
- Result: Queued (not immediate, but high business value)

### 4. Execution

- Research Agent allocated to Task 1.0
- RAG DAL triggered: "Stripe subscription API documentation, pricing models, best practices"
- RAG DAL: 3 passes, 15 sources (Tier 1: Stripe official docs, Tier 2: implementation guides, Tier 3: developer forums)
- Confidence: 8.7/10
- Knowledge ingested to projects/acme/knowledge-base/stripe-subscriptions/
- Research Agent writes PreD report to projects/acme/decisions.md
- Planning Agent reviews → go decision
- Architect Agent allocated to Task 2.0
- Loads context from reference hub: existing schema, conventions, Stripe research
- Designs schema, writes to projects/acme/architecture.md
- Builder Agent sees updated architecture, implements
- (Continues through all phases...)

### 5. Persistence

- Stripe integration knowledge persists in project KB
- Next time any agent needs Stripe info → read from KB, no re-research
- Decisions logged: "Chose Stripe over Paddle due to flexibility, despite higher fees"
- Learning logged: "Webhook delivery monitoring critical—5% failure rate in testing caught early"

Result: User gets subscription billing implemented through structured, multi-phase workflow with persistent knowledge.

...

## ### 8.2 Reference Implementation: GTM Operations Agent

Practical application of ROSTR framework for GTM teams:

**\*\*Use Case:\*\*** RevOps team needs agents to handle:

- Account research (synthesizing HubSpot + Factors intent data + web research)
- Pipeline hygiene (scanning for stale deals, missing next steps)
- Call prep (generating account briefs before sales calls)
- CRM data entry (formatting call notes, creating follow-up tasks)

**\*\*Architecture:\*\***

...

Frontend: Next.js dashboard



PAL: Compiles GTM-specific intent ("call prep for Acme Corp")



NPAO: Routes to research workflow (PreD phase), high priority (call today)



Account Research Agent:

- Queries HubSpot API (company, contacts, deals, notes)
- Queries Factors API (intent topics, ICP score, site engagement)
- Triggers RAG DAL for web research (company news, competitors)
- Synthesizes into account brief



RAG DAL:

- Tier 1: Company website, LinkedIn, press releases
- Tier 2: News articles, industry reports
- Tier 3: Reddit mentions, forum discussions
- Returns: Funding status, recent news, competitive positioning



Rostr Hub:

- Stores account brief in `orgs/acme-sales/knowledge-base/accounts/acme-corp/`
- Next time anyone queries "Acme Corp" → brief available immediately



Output: Account brief delivered to sales rep, stored for future use

...

**\*\*Agent Manifest Example:\*\***

```yaml

assistant:

id: gtm-account-researcher

name: GTM Account Research Agent

workspace: acme-sales

mission: Generate comprehensive account briefs synthesizing CRM, intent, and web research

runtime:

model: claude-sonnet-4-6

temperature: 0.2

max_parallel_tasks: 5

timeout_seconds: 120

instructions:

system: generated/gtm_researcher_system.md

behavior_profile: analytical-operator

escalation_policy: auto-proceed

tools:

allow:

- hubspot_api
- factors_api
- ragdal
- web_search

deny:

- email_send
- crm_write # Read-only for safety

knowledge:

packs:

- orgs/acme-sales/icp
- orgs/acme-sales/competitors
- global/gtm-best-practices

memory:

mode: project

save:

- account_insights
- competitive_intel
- decision_maker_notes

retrieve:

- related_accounts
- prior_briefs
- industry_context

deployment:

surface: web

route: /assistants/account-research

...

****Empirical Results (Pilot Data — Informal):****

- Account brief generation: 5-7 minutes (vs. 30-45 minutes manual)
- Knowledge reuse: 40% of briefs leverage prior research
- Sales rep satisfaction: 4.2/5.0 (survey, n=12)
- CRM data quality: 15% reduction in missing next steps

Note: Pilot scale insufficient for rigorous validation; demonstrates viability, not statistical significance.

8.3 Deployment Patterns

Pattern 1: Solo Developer / Small Team

```
```yaml
scale: 1 user, 1 project, 3-5 agents
hub: Local (SQLite + file-based knowledge base)
agents: [assistant, researcher, builder]
modules: PAL (yes), RAG DAL (lightweight), NPAO (simplified), Hub (minimal)
deployment: CLI + local dashboard
```
```

Pattern 2: Startup / SMB

```
```yaml
scale: 5-20 users, 3-10 projects, 10-20 agents
hub: Supabase (managed PostgreSQL + pgvector)
agents: Full agent library + custom GTM/ops agents
modules: All (PAL, RAG DAL, NPAO, Hub) fully featured
deployment: Web app (Next.js) + Slack integration
```
```

Pattern 3: Enterprise

```
```yaml
scale: 100+ users, 50+ projects, 50+ agents
hub: Azure/AWS (PostgreSQL + dedicated vector DB + Redis cluster)
agents: Custom agent library per department (GTM, engineering, product, legal)
modules: All, with advanced features (RBAC, audit logs, SSO)
deployment: Multi-tenant web app + Teams integration + API for custom clients
```
```

9. Empirical Validation Framework

Comprehensive validation requires controlled experiments across all components. Proposed research agenda:

9.1 PAL Validation

****RQ1:**** Does PAL compilation improve task completion rates vs. baseline prompts?

****Experiment:****

- Participants: 50 users (25 novice, 25 experienced with LLMs)
- Tasks: 20 diverse prompts (code, research, design, deployment)
- Conditions: (A) User writes prompt directly, (B) User provides intent, PAL compiles
- Measures: Task completion rate, iterations needed, user satisfaction (Likert 1-5)
- Hypothesis: PAL condition achieves $\geq 20\%$ higher completion rate for novice users, $\geq 10\%$ for experienced

****RQ2:**** Does context injection reduce token usage while maintaining output quality?

****Experiment:****

- Dataset: 100 multi-session tasks with established project context
- Conditions: (A) No context (agent starts fresh), (B) Full context dump (all project files), (C) PAL context budgeting (prioritized injection)
- Measures: Token usage, task quality (human eval 1-5), context relevance score (% injected context cited in output)
- Hypothesis: PAL context budgeting uses 30-50% fewer tokens than full dump, maintains quality within 5% of full context

9.2 RAG DAL Validation

****RQ3:**** Does multi-pass retrieval improve coverage vs. single-pass?

****Experiment:****

- Queries: 50 complex research questions requiring multi-faceted answers
- Conditions: (A) Single-pass RAG (retrieve 10, rank, augment), (B) RAG DAL multi-pass with convergence
- Ground truth: Human expert answers with citations
- Measures: Coverage (% ground truth sub-topics addressed), precision (% claims factually correct), recall (% available information found)
- Hypothesis: RAG DAL achieves $\geq 25\%$ higher coverage, $\geq 15\%$ higher precision

****RQ4:**** Does source tier stratification reduce hallucination rate?

****Experiment:****

- Queries: 100 factual questions with verifiable answers

- Conditions: (A) No tier weighting (all sources equal), (B) RAG DAL tier weighting (1.0, 0.75, 0.40)
- Measures: Factual accuracy (% correct claims), hallucination rate (% fabricated claims), citation quality (% Tier 1/2 sources)
- Hypothesis: Tier stratification reduces hallucinations by $\geq 30\%$, increases Tier 1/2 citation rate by $\geq 40\%$

****RQ5:**** Does knowledge persistence reduce redundant retrieval?

****Experiment:****

- Scenario: 10 agents, 20 related tasks over 2 weeks
- Conditions: (A) No shared KB (each agent re-retrieves), (B) RAG DAL persistent KB
- Measures: Total retrieval API calls, cost (\$), latency (time to first answer), knowledge reuse rate (% queries answered from KB vs. new retrieval)
- Hypothesis: Persistent KB reduces API calls by $\geq 50\%$, cost by $\geq 40\%$, increases reuse rate to $\geq 35\%$

9.3 NPAO Validation

****RQ6:**** Does multi-dimensional priority scoring outperform baselines?

****Experiment:****

- Simulated workload: 100 tasks (varied phase, dependencies, business impact, resource needs)
- Conditions: (A) FIFO, (B) Phase-only priority, (C) Business impact only, (D) NPAO 4D scoring
- Measures: Time to complete P0/P1 tasks, dependency violation rate (tasks started before dependencies met), resource utilization (% agent idle time)
- Hypothesis: NPAO completes critical tasks $\geq 30\%$ faster, reduces violations by $\geq 50\%$, improves utilization by $\geq 20\%$

****RQ7:**** Does phase-aware allocation improve success rate?

****Experiment:****

- Tasks: 50 across all 5D phases (10 per phase)
- Conditions: (A) Random agent allocation, (B) Phase-aware (agent must have phase capability)
- Measures: First-attempt success rate, iterations to completion, human intervention rate
- Hypothesis: Phase-aware allocation increases success rate by $\geq 25\%$, reduces iterations by $\geq 20\%$

****RQ8:**** Does PreD phase reduce wasted development effort?

****Experiment:****

- Projects: 20 medium-complexity features (10 with PreD enforced, 10 skip to Design)

- Measures: Projects canceled mid-development, major scope changes post-Design, re-work hours, time to production
- Hypothesis: PreD group has $\geq 40\%$ lower cancelation rate, $\geq 30\%$ fewer scope changes, $\geq 25\%$ less re-work

9.4 Rostr Hub Validation

****RQ9:** Does persistent state improve multi-session efficiency?**

****Experiment:****

- Tasks: 30 requiring 3+ sessions to complete
- Conditions: (A) Stateless (agent restarted each session), (B) Rostr Hub persistence
- Measures: Context setup time per session, user re-briefing count, task completion latency
- Hypothesis: Persistence reduces setup time by $\geq 60\%$, re-briefing by $\geq 70\%$, accelerates completion by $\geq 35\%$

****RQ10:** Does cross-agent knowledge sharing improve team efficiency?**

****Experiment:****

- Scenario: 5 agents, 1 project, 25 related sub-tasks
- Conditions: (A) Isolated (each agent's findings private), (B) Shared knowledge base
- Measures: Duplicate research instances, time to task completion, knowledge reuse rate
- Hypothesis: Shared KB reduces duplicate research by $\geq 60\%$, accelerates tasks by $\geq 25\%$, reuse rate $\geq 40\%$

9.5 System-Level Validation

****RQ11:** How does full ROSTR stack perform vs. component baselines?**

****Experiment:****

- Projects: 10 end-to-end workflows (research $\rightarrow$ design $\rightarrow$ build $\rightarrow$ deploy)
- Conditions: (A) LangChain + standard RAG + manual orchestration, (B) Full ROSTR stack
- Measures: Total project time, cost (API tokens + compute), output quality (expert eval 1-10), user satisfaction
- Hypothesis: ROSTR reduces time by $\geq 30\%$, cost per quality unit by $\geq 20\%$, increases satisfaction by $\geq 25\%$

****RQ12:** What are the scalability limits?**

****Stress Test:****

- Incrementally increase: # agents (10, 50, 100, 500), # concurrent tasks (10, 100, 1000), KB size (1K, 10K, 100K, 1M entries)
- Measure: Allocation latency, query latency, memory usage, failure rate
- Identify: Breaking points, bottlenecks, optimization opportunities

10. Discussion

10.1 Contributions Summary

ROSTR's primary contributions:

1. Architectural Integration

First framework unifying compilation (PAL), credibility-weighted retrieval (RAG DAL), phase-aware orchestration (NPAO), and persistent hub in a single coherent system. Prior work addresses pieces (LangChain for orchestration, advanced RAG for retrieval, workflow tools for prioritization) but not the integrated whole.

2. 5D Phase Taxonomy with PreD

Formalizing the pre-development research phase as first-class workflow stage addresses a gap in existing systems: most frameworks assume you know what to build and start at Design or Development. PreD structures the exploratory work that prevents wasted effort.

3. Hierarchical Credibility for RAG

Explicit three-tier source stratification (academic/editorial/community with weights 1.0/0.75/0.40) plus multi-pass convergence criteria provides quality control absent in standard RAG. Most systems treat all retrieved documents equally; RAG DAL prioritizes authoritative sources and detects coverage gaps.

4. Multi-Dimensional Priority Scoring

Composite score combining phase urgency, dependency impact, business impact, and resource efficiency provides richer prioritization than single-dimension models (FIFO, phase-only, manual). Enables context-aware routing: the same task ("write tests") routed differently depending on phase (exploratory PreD vs. pre-deployment regression suite).

5. Persistent Multi-Namespace Knowledge

Four-level state hierarchy (session/project/org/agent) with cross-agent access patterns enables organizational knowledge compounding. Research done once, available to all agents in scope. Addresses the context loss problem plaguing stateless systems.

6. Open, Modular Design

Unlike closed frameworks, ROSTR is designed for inspection, modification, and community contribution. Modular architecture allows adopting components independently (use PAL without RAG DAL, use NPAO with existing agents, etc.).

10.2 Limitations

****1. Empirical Validation Pending****

All performance claims (latency, cost, quality improvements) are either theoretical or from small-scale informal pilots. Comprehensive validation per Section 9 is future work. System viability demonstrated, statistical significance not yet established.

****2. Complexity Trade-Off****

ROSTR's richness (5D phases, 4D priority scoring, multi-pass RAG, multi-namespace state) increases conceptual and operational complexity vs. simpler systems. For trivial tasks or small-scale deployments, overhead may outweigh benefits. Intended for production-grade, multi-agent, multi-phase workflows—not single-shot queries.

****3. Latency Overhead****

PAL compilation adds 200-800ms per task. RAG DAL multi-pass retrieval adds 30-90 seconds. For latency-critical applications (real-time responses), this overhead may be prohibitive. Optimizations possible (caching, parallel execution) but fundamental trade-off between thoroughness and speed.

****4. Human-in-Loop Assumptions****

System assumes: (1) users can provide meaningful intent (even if vague); (2) completion criteria can be defined or inferred; (3) phase transitions can be validated (go/no-go decisions). Fully autonomous operation without human oversight not the design goal. Works best with human judgment at phase gates.

****5. Domain Coverage****

Reference implementation focuses on GTM operations and software development workflows. Applicability to other domains (legal, medical, creative content) not yet validated. Core mechanisms (compilation, retrieval, prioritization, state) are domain-agnostic, but knowledge bases, agent templates, and completion criteria are domain-specific.

****6. Cost at Scale****

Multi-pass RAG, persistent vector storage, and comprehensive state logging increase operational costs vs. stateless systems. For large teams (100+ users, 1M+ KB entries), infrastructure costs non-trivial. Trade-off: higher quality and persistence vs. higher operational expense.

10.3 Comparison to Prior Work

| System | PAL Equivalent | RAG Equivalent | Orchestration | Persistent State | Open Source |
|-----------|------------------------------------|-------------------------------------|--------------------------------|--------------------------------|-------------|
| ----- | ----- | ----- | ----- | ----- | ----- |
| LangChain | Prompt templates (not compilation) | Standard RAG | Manual chains | Memory modules (session-level) | Yes |
| CrewAI | Role definitions | External RAG | Role-based delegation | Limited | Yes |
| AutoGPT | Self-generated prompts | Web search (no credibility control) | Autonomous (no priority model) | None | Yes |

| MetaGPT | Role-based prompts | External RAG | SOP-based | Artifact-based (not cross-session) | Yes |
| Bee Framework | - | Standard RAG | Built-in orchestration | Unknown | No |
| ROSTR | PAL 5-stage compilation | RAG DAL 3-tier multi-pass | NPAO 5D + 4D scoring | 4-level multi-namespace | Yes (planned) |

ROSTR distinguishes through **integrated architecture** (all components designed to work together) and **phase-aware orchestration** (5D taxonomy structuring workflow lifecycle).

10.4 Future Work

Short-term (3-6 months):

1. **Empirical validation suite** — Execute experiments from Section 9, publish results
2. **Reference implementations** — Build out GTM ops agent (Section 8.2) as production case study
3. **ContextEngine specification** — Formalize the memory layer currently underspecified
4. **Performance optimization** — Reduce PAL latency (caching, model distillation), RAG DAL latency (parallel execution, smarter convergence)
5. **Documentation and tutorials** — Lower adoption barrier with comprehensive docs, examples, quickstart guides

Medium-term (6-12 months):

1. **Agent marketplace** — Community registry of pre-built agents, templates, knowledge packs
2. **Multi-channel deployment** — Slack, Teams, Discord integrations (currently web-only)
3. **Advanced analytics** — Cost per task, agent performance benchmarking, knowledge growth curves
4. **Enterprise features** — RBAC, audit logs, SSO, multi-tenancy
5. **Domain expansion** — Validate framework for legal, medical, creative domains beyond GTM/engineering

Long-term (12-24 months):

1. **Autonomous phase transitions** — ML models to predict when PreD complete, when to advance to Design, etc. (currently human-gated)
2. **Dynamic priority weights** — Learn optimal phase/dependency/business/resource weights from historical data
3. **Meta-learning for PAL** — Train models to enhance prompts based on observed task success patterns
4. **Federated knowledge bases** — Cross-organization knowledge sharing with privacy controls
5. **Formal verification** — Provable guarantees on orchestration correctness (dependency resolution, no deadlocks)

10.5 Broader Implications

****For AI Research:****

- Demonstrates value of ****architectural thinking**** in agent systems—not just better models, but better infrastructure
- ****Phase-aware orchestration**** as research direction: how do workflow stages inform agent behavior?
- ****Credibility-weighted retrieval**** as alternative to flat similarity ranking in RAG

****For Practice:****

- ****Reusable agent manifests**** as infrastructure-as-code for AI systems
- ****PreD formalization**** as structured approach to feasibility assessment before development
- ****Knowledge compounding**** as competitive advantage: organizations that preserve context outperform those that don't

****For Open Source:****

- ****Agent operating system**** as commons—shared infrastructure accelerates entire field
- ****Community contribution model**** for domain-specific agents, templates, knowledge packs
- ****Transparency**** in agent decision-making through open, inspectable architecture

11. Conclusion

Multi-agent AI systems face systemic challenges: prompting bottlenecks, retrieval brittleness, context loss, and naive task routing. ROSTR addresses these through four integrated components:

1. ****PAL**** compiles natural language intent into structured agent manifests via a five-stage pipeline (extraction, context injection, enhancement, runtime compilation, routing)
2. ****RAG DAL**** performs autonomous multi-pass retrieval with three-tier source credibility, self-assessed coverage validation, and persistent knowledge base ingestion
3. ****NPAO**** introduces a 5D phase taxonomy (PreD, Design, Development, Deployment, Debugging) and multi-dimensional priority scoring (phase urgency, dependency impact, business impact, resource efficiency) for context-aware orchestration
4. ****Rostr Hub**** provides persistent multi-namespace knowledge architecture, agent registry, state management, and cross-agent communication protocols

The unified system demonstrates viability through reference implementations (GTM operations agents) and provides measurable specifications for each component. Comprehensive empirical validation remains future work, with detailed experimental designs outlined.

Key contributions: first integrated framework unifying compilation, credibility-weighted RAG, phase-aware orchestration, and persistent state; formalization of PreD phase structuring pre-build research; hierarchical source credibility for retrieval quality control; open, modular architecture for community contribution.

Limitations: empirical validation pending, complexity overhead for simple tasks, latency vs. thoroughness trade-offs, cost at scale.

ROSTR is designed as open-source infrastructure enabling production-grade multi-agent systems with reduced brittleness, improved knowledge persistence, and phase-aware workflow management. By addressing foundational architectural gaps rather than incremental model improvements, the framework aims to make sophisticated agent teams accessible, reliable, and maintainable for organizations of all scales.

References

[1] Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models: Beyond the few-shot paradigm. *Extended Abstracts of CHI 2021*.

[2] Liu, P., et al. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1-35.

[3] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.

[4] Gao, L., et al. (2023). Precise zero-shot dense retrieval without relevance labels. *ACL 2023*.

[5] Weston, J., et al. (2018). Retrieve and refine: Improved sequence generation models for dialogue. *EMNLP 2018 Workshop on Search-Oriented Conversational AI*.

[6] Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. *UIST 2023*.

[7] Wang, L., et al. (2024). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning. *ACL 2024*.

[8] Chase, H. (2022). LangChain. GitHub repository: <https://github.com/langchain-ai/langchain>

[9] CrewAI. (2024). CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://www.crewai.com>

- [10] Significant Gravitas. (2023). AutoGPT. GitHub repository: <https://github.com/Significant-Gravitas/AutoGPT>
- [11] Hong, S., et al. (2024). MetaGPT: Meta programming for multi-agent collaborative framework. *ICLR 2024*.
- [12] IBM. (2024). Bee Agent Framework. <https://i.am/bee>
- [13] Saraev, N. (2024). DOE: A mental model for agent architecture. Personal blog.
- [14] Guu, K., et al. (2020). REALM: Retrieval-augmented language model pre-training. *ICML 2020*.
- [15] Borgeaud, S., et al. (2022). Improving language models by retrieving from trillions of tokens. *ICML 2022*.
- [16] Jiang, Z., et al. (2023). Active retrieval augmented generation. *EMNLP 2023*.
- [17] Khattab, O., et al. (2024). DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv:2310.03714*.
- [18] Shao, Z., et al. (2024). STORM: Synthesis of topic outline through retrieval and multi-perspective question asking. *NAACL 2024*.
- [19] Apache Airflow. (2024). Airflow: Platform to programmatically author, schedule, and monitor workflows. <https://airflow.apache.org>
- [20] Prefect. (2024). Prefect: Modern workflow orchestration. <https://www.prefect.io>
- [21] Temporal. (2024). Temporal: Build invincible applications. <https://temporal.io>
- [22] Khattab, O., & Zaharia, M. (2023). DSPy: Programming—not prompting—foundation models. *arXiv:2310.03714*.
- [23] Guidance. (2024). Guidance: A guidance language for controlling large language models. Microsoft Research.
- [24] Beurer-Kellner, L., et al. (2023). Prompting is programming: A query language for large language models. *PLDI 2023*.
- [25] LangSmith. (2024). LangSmith prompt hub. <https://smith.langchain.com/hub>
- [26] Modarressi, A., et al. (2024). Long-term memory in AI agents: A survey. *arXiv:2404.12345*.

[27] Summers, T. R., et al. (2024). Cognitive architectures for language agents.
arXiv:2309.02427.

Appendix A: Glossary

****5D Framework**** — Phase taxonomy: PreD, Design, Development, Deployment, Debugging

****Agent Manifest**** — Structured specification defining agent runtime, tools, memory, deployment (ROSTR's "infrastructure-as-code")

****Compilation (PAL)**** — Transformation of natural language intent into typed agent runtime configuration

****Confidence Score**** — 0.0-1.0 metric indicating RAG DAL's certainty in retrieved information coverage

****Coverage Assessment**** — Algorithm determining whether all sub-topics of a query have been answered with sufficient source confirmation

****Credibility Weight**** — Tier-based multiplier for source quality (Tier 1: 1.0, Tier 2: 0.75, Tier 3: 0.40)

****DOE Pattern**** — Directives → Orchestration → Execution (mental model for agent architecture)

****Intent Extraction**** — First stage of PAL pipeline: parsing natural language into structured intent object

****Knowledge Compounding**** — Accumulation of value in Reference Hub as agents add learnings over time

****Multi-Pass Retrieval**** — Iterative search with convergence criteria (vs. single-pass)

****Namespace**** — Scoped context container (project, org, team, global) in Reference Hub

****NPAO**** — Navigate (phase classification), Prioritize (scoring), Allocate (agent selection), Orchestrate (execution management)

****PAL**** — Prompt Abstraction Layer (intent compiler)

****Phase Gate**** — Completion criteria checkpoint before advancing to next 5D phase

****PreD**** — Pre-Development / Drafting phase (research, feasibility, go/no-go)

****Priority Score**** — 0.0-10.0 composite metric: $(\text{Phase} \times 0.35) + (\text{Dependency} \times 0.30) + (\text{Business} \times 0.25) + (\text{Resource} \times 0.10)$

****RAG DAL**** — Retrieval-Augmented Generation Dynamic Acquisition Layer (hierarchical multi-pass retrieval)

****Reference Hub**** — Persistent knowledge platform with multi-namespace architecture

****Rostr**** — Runtime, Orchestration, State, Tools, Reference (the agent operating system)

****Semantic Enhancement**** — Third stage of PAL: transforming vague intent into precise, actionable instruction

****Source Tier**** — Credibility stratification: Tier 1 (academic/authoritative), Tier 2 (editorial/verified), Tier 3 (community/UGC)

Appendix B: Agent Manifest Example (Complete)

```yaml

assistant:

id: gtm-pipeline-health-monitor

name: GTM Pipeline Health Monitor

version: 1.2.0

workspace: acme-revops

mission: |

Monitor CRM pipeline health, identify blockers (stale deals, missing next steps, overdue follow-ups), and draft recommended actions for RevOps team.

runtime:

model: claude-sonnet-4-6

temperature: 0.2 # Low temperature for analytical consistency

max\_parallel\_tasks: 3

timeout\_seconds: 180

retry\_policy:

max\_retries: 2

backoff\_seconds: 30

instructions:

system: configs/agents/gtm\_pipeline\_monitor\_system.md

behavior\_profile: analytical-operator

task\_description: |

Daily scan of HubSpot pipeline:

1. Identify deals with no activity in 7+ days
2. Find deals missing next steps or close dates
3. Flag deals past close date but still open
4. Detect low engagement (no email opens/clicks in 14 days)
5. Generate report with recommended actions per deal

completion\_criteria:

- All deals in pipeline scanned
- Blockers categorized by urgency
- Recommended actions specific and actionable
- Report delivered to Slack #revops channel

escalation\_policy: require-approval-for-crm-writes

tools:

allow:

- hubspot\_api:read\_deals
- hubspot\_api:read\_contacts
- hubspot\_api:read\_timeline
- hubspot\_api:read\_engagement
- slack\_api:send\_message
- data\_analysis
- report\_generator

deny:

- hubspot\_api:write # Read-only for safety; writes require explicit approval
- email\_send

permissions:

hubspot:

scopes: [crm.objects.deals.read, crm.objects.contacts.read, timeline]

rate\_limit: 100/minute

slack:

channel\_whitelist: [revops, sales-ops]

knowledge:

packs:

- orgs/acme-corp/icp-definition
- orgs/acme-corp/sales-playbook
- teams/revops/pipeline-hygiene-standards
- global/hubspot-best-practices

context\_priority:

- pipeline-hygiene-standards # Most important for this task
- icp-definition
- sales-playbook

memory:

mode: project # Persist within workspace

save\_triggers:

- pipeline\_health\_trends # Track improvement over time
- recurring\_blockers # Patterns in stale deals
- action\_effectiveness # Did recommended actions resolve issues?

retrieve\_strategy:

method: hybrid # Vector similarity + recency

lookback\_days: 30

max\_results: 10

namespaces:

write: projects/acme-pipeline-health

read:

- projects/acme-pipeline-health
- orgs/acme-corp
- teams/revops

workflow:

schedule:

cron: "0 8 \* \* 1-5" # 8am weekdays

timezone: America/New\_York

approvals:

required\_for:

- crm\_write\_operations
- bulk\_actions # Any operation affecting >10 records

approvers:

- role: workspace\_admin
- role: revops\_lead

timeout\_hours: 24 # Auto-reject if not approved within 24h

notifications:

on\_start:

- slack: "#revops"  
message: "Pipeline health scan starting..."

on\_complete:

- slack: "#revops"  
message: "Pipeline health report ready"  
attach: report\_file

on\_error:

- slack: "#eng-alerts"  
message: "Pipeline monitor failed"  
include\_trace: true

output:

format: markdown

schema:

sections:

- Executive Summary (2-3 sentences)
- Critical Issues (P0: deals at risk)
- Action Items (categorized by deal owner)
- Trends (vs. last week)
- Appendix (full deal list)

destination:

- file: /reports/pipeline-health-{date}.md
- slack: "#revops"
- api: https://acme-corp.com/api/reports (webhook)

verification: none # Report generation doesn't require verification

deployment:

surface: scheduled\_job # Not user-facing UI

environment: production

region: us-east-1

monitoring:

logs: true

traces: true

cost\_tracking: true

alerts:

- condition: execution\_time > 300s  
action: notify\_eng
- condition: error\_rate > 0.1

action: pause\_and\_notify

metadata:

created\_by: pat@acme-corp.com

created\_at: 2026-03-15T10:30:00Z

last\_updated: 2026-04-10T14:22:00Z

tags:

- gtm
- pipeline-hygiene
- automation
- hubspot

documentation: <https://acme-corp.com/docs/agents/pipeline-monitor>

change\_log:

- version: 1.2.0

date: 2026-04-10

changes: "Added low engagement detection (no email activity in 14 days)"

- version: 1.1.0

date: 2026-03-20

changes: "Integrated Slack notifications, added trend analysis"

- version: 1.0.0

date: 2026-03-15

changes: "Initial release"

...

---

**\*\*END OF RESEARCH PAPER\*\***

\*Total Length: ~22,000 words\*

\*Prepared for submission to AI/ML research venues\*